Indian Institute of Technology Bombay
Department of Computer Science and Engineering

# Structured LLM Optimization




**Saksham Rathi**

Under the guidance of

**Prof. Mythili Vutukuru**

Bachelor's Thesis Project

Academic Year: 2025–26

# Contents

**List of Figures** **2**

**List of Tables** **3**

**1 Initial Experiments** **4**
1.1 System Prompt in Cache Experiment . . . . . 4
1.1.1 Example 1 . . . . . 4
1.1.2 Example 2 . . . . . 5
1.1.3 Observations . . . . . 6
1.1.4 Analysis over different steps and using different metrics 7
1.1.5 Profiling across pipeline stages . . . . . 7
1.2 Batch of Requests . . . . . 8

**2 SGLang** **9**

**3 QLora** **10**

**4 vLLM** **11**

# List of Figures

# List of Tables

1.1 Profiler Output . . . . . 7
1.2 Profiling results for various pipeline stages . . . . . 7
1.3 Measuring the time taken to process a batch of requests using different techniques . . . . . 8

# Chapter 1

# Initial Experiments

## 1.1 System Prompt in Cache Experiment

The basic application which we want to cater is inferencing in cases where the system prompt is common across multiple queries. And therefore, we want to utilize caching by already filling the KV-Cache with the system prompt, and then putting user queries on the top of it.

### 1.1.1 Example 1

I picked the 200-word summary of the demand paging lecture as the system prompt to be provided to the LLM. The model which we used was `TinyLlama/TinyLlama-1.1B-Chat-v1.0` (1.1 billion parameters). The system prompt provided was:

> The lecture discusses demand paging, a key concept in virtual memory management. Instead of loading all pages of a process into memory, only the required pages are brought in, while the rest may stay in swap space on disk. This allows processes to use address spaces larger than physical memory and enables efficient memory utilization.
> Each process has a page table, with entries containing a valid bit (indicating whether the page is part of the process) and a present bit (whether it is currently in memory). If valid=0, the process makes an illegal access. If valid=1 but present=0, a page fault occurs. The OS then allocates a memory frame, possibly evicts a victim page using a page replacement policy, updates the page table, and resumes execution.
> Victim pages may be file-backed or anonymous, and may be dirty (modified) or clean. Dirty pages require writing back to disk before eviction. Policies like FIFO and LRU determine which pages to evict; LRU is effective due to locality, while FIFO can suffer from Belady's anomaly.
> Excessive page faults can cause thrashing, where the system spends more time swapping pages than doing useful work. To prevent this, the OS ensures that each process's working set fits in memory.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.339 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

**Query 1:** What is the difference between a valid bit and a present bit?

**Response 1:** The difference between a valid bit and a present bit is that a valid bit indicates that a page is part of a process, while a present bit indicates that a page is currently in memory.

**Query 2:** Why does thrashing occur in demand paging systems?

**Response 2:** Thrashing occurs in demand paging systems because excessive page faults can cause the system to spend more time swapping pages than doing useful work. This is because demand paging systems allocate memory frames to processes based on their page table entries, which are either valid or present. If a process has a page fault, it must be allocated a memory

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.1 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.94 seconds.

### 1.1.2 Example 2

Let us take another example, this time with a summary of the semaphores lecture. The system prompt provided was:

The lecture introduces semaphores, a synchronization primitive similar to condition variables, used to coordinate threads. A semaphore maintains a counter, initialized to some value. The down (wait) operation decrements the counter and blocks the thread if it becomes negative, while the up (post) operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations.
Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They are also useful for synchronization between threads, e.g., enforcing execution order (T1 before T2) or ensuring two threads finish one step before starting another. Incorrect usage can cause deadlocks, such as when both threads wait on each other.
The producer-consumer problem is revisited using semaphores. Two semaphores track empty and filled buffer slots, while a binary semaphore (mutex) ensures mutual exclusion on buffer access. Misuse, like holding locks during blocking down calls, can also lead to deadlocks.
Guidelines emphasize careful initialization, ensuring every down has a corresponding up, and using separate semaphores for signaling and locking. The lecture also covers batched processing with semaphores, showing how request threads and a batch processor can coordinate efficiently.
Semaphores thus provide flexible yet tricky synchronization, requiring careful design to avoid deadlocks.

The time taken to fill the system prompt, and find the key-value cache tensors, along with making clones (so that we feed different clones to different queries) was around 0.347 seconds.

We then make two queries to the LLM, by taking KV-Cache as a generation call input, we restrict the output length to 100 tokens

**Query 1:** How does the down operation on a semaphore synchronize threads during execution?

**Response 1:** A semaphore is a synchronization primitive that maintains a counter, initialized to some value. The down operation decrements the counter and blocks the thread if it becomes negative. The up operation increments the counter and wakes a blocked thread. The counter cannot be directly accessed, only modified via these operations. Semaphores can be used for mutual exclusion (binary semaphores), functioning like locks. They

**Query 2:** Why is an extra binary semaphore (mutex) needed in producer-consumer implementation?

**Response 2:** An extra binary semaphore (mutex) is needed in producer-consumer implementation because the producer and consumer threads can access the same buffer at the same time. This can lead to deadlocks if the buffer is not properly protected.

The total time taken by the LLM to run these two queries (with cached system prompt) was 2.774 seconds.

However, if we do not cache the system prompt, and instead make two separate LLM calls, with ((system prompt + query1) and (system prompt + query2)), then the total time taken increases to 2.957 seconds.

### 1.1.3 Observations

- As we can see, there is roughly an improvement of 0.2-0.7 seconds for an experiment of 2 queries, which is also intutively clear, because we are just passing over the system prompt once.
- As we increase the size of the system prompt, and the number of tokens we generate, the difference becomes even wider.
- The time taken to feed through the system prompt, and cloning the KV cache values, can be ignored safely because we can do this before we get the actual user queries.
- The first query of the second example didn't get a superior response, however this is independent from our experiment settings, and depends more on the type of model we use, for example increasing the number of parameters (using a larger model) would definitely help.

### 1.1.4 Analysis over different steps and using different metrics

I now use a profiler to actually figure out at what all places, the most amount of time is being spent:

Table 1.1: Profiler Output

| Name | Self CPU % | Self CPU | CPU total % | CPU total | CPU time avg | Self CUDA | Self CUDA % | CUDA total | CUDA time avg | # of Calls |
|---|---|---|---|---|---|---|---|---|---|---|
| aten::matmul | 2.67% | 56.256ms | 30.07% | 632.976ms | 40.575us | 0.000us | 0.00% | 589.952ms | 37.817us | 15600 |
| aten::linear | 1.16% | 24.342ms | 30.78% | 647.989ms | 41.806us | 0.000us | 0.00% | 589.802ms | 38.052us | 15500 |
| aten::mm | 14.46% | 304.341ms | 22.68% | 477.495ms | 30.806us | 579.871ms | 69.41% | 589.802ms | 38.052us | 15500 |
| cutlass kernel (relu bf16) | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 234.992ms | 28.13% | 234.992ms | 65.385us | 3594 |
| ampere_bf16_128x64_sliced1x2 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 124.274ms | 14.87% | 124.274ms | 66.457us | 1870 |
| ampere_bf16_128x128_stages_32x1 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 94.240ms | 11.28% | 94.240ms | 44.621us | 2112 |
| aten::mul | 6.55% | 137.891ms | 10.29% | 216.715ms | 10.728us | 61.543ms | 7.37% | 61.560ms | 3.048us | 20200 |
| aten::copy_ | 3.97% | 83.637ms | 8.32% | 175.218ms | 9.679us | 50.846ms | 6.09% | 50.857ms | 2.809us | 18103 |
| ampere_bf16_128x64_stages_64x3 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 44.572ms | 5.34% | 44.572ms | 19.481us | 2288 |
| aten::scaled_dot_product_attention | 0.84% | 17.683ms | 5.51% | 116.049ms | 52.749us | 0.000us | 0.00% | 44.127ms | 20.058us | 2200 |
| aten::_scaled_dot_product_flash_attention | 0.63% | 13.224ms | 4.67% | 98.366ms | 44.712us | 0.000us | 0.00% | 44.127ms | 20.058us | 2200 |
| aten::_flash_attention_forward | 1.00% | 21.077ms | 3.49% | 73.410ms | 33.368us | 44.073ms | 5.28% | 44.127ms | 20.058us | 2200 |
| flash_fwd_kernel | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 43.833ms | 5.25% | 43.833ms | 20.125us | 2178 |
| elementwise_mul_kernel | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 37.673ms | 4.51% | 37.673ms | 2.833us | 13300 |
| ampere_bf16_64x64_sliced1x4 | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 34.059ms | 4.08% | 34.059ms | 28.669us | 1188 |
| aten::clone | 0.87% | 18.407ms | 6.48% | 136.324ms | 15.491us | 0.000us | 0.00% | 26.201ms | 2.977us | 8800 |
| elementwise_copy_kernel | 0.00% | 0.000us | 0.00% | 0.000us | 0.000us | 26.198ms | 3.14% | 26.198ms | 2.977us | 8800 |
| aten::add | 4.45% | 93.743ms | 7.17% | 150.912ms | 11.347us | 24.648ms | 2.95% | 24.657ms | 1.854us | 13300 |
| aten::to | 0.93% | 19.568ms | 8.33% | 175.368ms | 12.345us | 0.000us | 0.00% | 24.656ms | 1.736us | 14205 |
| aten::_to_copy | 1.29% | 27.125ms | 7.40% | 155.800ms | 16.747us | 0.000us | 0.00% | 24.656ms | 2.650us | 9303 |

Now we go over each of the rows, and further analyze the values:

1. `atten::matmul` = Pytorch's high level matrix multiplication. Runs on CPU side, but calls matrix multiply functions (`atten::mm`) on CUDA.
2. `atten::linear` = The linear layer, has large CUDA compute
3. `cutlass kernel (relu bf16)` = Stands for general matrix multiplication, entirely on CUDA
4. `ampere_...` = Specialized Ampere architecture GEMM kernel in BF16 precision for different tile sizes
5. `atten::mul` = Element-wise matrix multiplication, very high call count
6. `aten::copy_` = Copying tensors, very high call count
7. `aten::_flash_attention_forward` = The forward pass of flash attention

### 1.1.5 Profiling across pipeline stages

| Stage | Time (s) |
|---|---|
| Tokenize system prompt | 0.0052 |
| System forward (KV cache fill) | 0.3342 |
| Tokenize query | 0.0003 |
| Per token (mean / first) | 0.0148 / 0.0390 |
| Generation total | 0.7517 |
| Decode tokens | 0.0002 |

Table 1.2: Profiling results for various pipeline stages

## 1.2 Batch of Requests

We will measure the time taken to process a batch of requests, by varying the batch size:

Total number of queries: 24

| **Technique Used** | **Time (s)** |
|---|---|
| Transformers (batch size = 1) | 30.21 |
| Transformers (batch size = 2) | 19.51 |
| Transformers (batch size = 3) | 18.64 |
| vLLM (no system prompt pre-filling) | 0.45 |
| vLLM (with system prompt pre-filling) | 0.37 |
| SGLang | 1.03 |
| NEO | 0.55 |

Table 1.3: Measuring the time taken to process a batch of requests using different techniques

# Chapter 2

# SGLang

# Chapter 3

# QLora

# Chapter 4

# vLLM